End-to-End Secure Message Delivery

II3230 - Keamanan Informasi

Disusun Oleh:
Wilson (18223012)
Derick Amadeus Budiono (18223090)

Sekolah Teknik Elektro dan Informatika
Program Studi Sistem dan Teknologi Informasi
Semester VI - 2025/2026
Institut Teknologi Bandung
2026

Daftar Isi

1. Pendahuluan **5**
1.1. Enkripsi Simetris 5
1.2 Enkripsi Asimetris 5
1.3 Fungsi Hash 5
1.4 Digital Signature 6
2. Tujuan Tugas **6**
2.1. Memahami Alur Keamanan Data End-to-End 6
2.2. Implementasi Kombinasi Komponen Kriptografi 6
2.3. Simulasi Komunikasi Antar Jaringan (IP) 7
2.4. Verifikasi dan Validasi Sisi Penerima 7
3. Desain Skenario **7**
3.1. Entitas dan Topologi Jaringan 7
3.2. Alamat IP dan Konektivitas 7
3.3. Alur Keamanan Data 7
4. Topologi & Arsitektur Sistem **8**
4.1. Arsitektur Jaringan 8
4.2. Model Komunikasi Client-Server 8
4.3. Komponen Keamanan End-to-End 8
5. Algoritma yang Diterapkan **9**
5.1 Enkripsi Simetris: AES-256 (Advanced Encryption Standard) 9
5.2 Enkripsi Asimetris: RSA-2048 (Rivest-Shamir-Adleman) 9
5.3 Fungsi Hash: SHA-256 (Secure Hash Algorithm) 10
5.4 Digital Signature: RSA-PSS (Probabilistic Signature Scheme) 11
6. Langkah Kerja & Implementasi **11**
6.1. Sisi Alice (Pengirim) 11
6.2. Sisi Bob (Penerima) 13
6.3. Format Payload 15
7. Hasil Pengujian & Analisis **15**
8. Kesimpulan **19**

Daftar Gambar

Gambar 5.1 Enkripsi simetris menggunakan AES.....9
Gambar 5.2 Enkripsi simetris menggunakan RSA..... 10
Gambar 5.2 Hash menggunakan RSA..... 11
Gambar 7.1 Pemantauan Permintaan Masuk..... 16
Gambar 7.2 GUI Pemantauan Permintaan Masuk.....16
Gambar 7.3 Pengiriman Pesan oleh Alice.....16
Gambar 7.4 GUI Pengiriman Pesan oleh Alice..... 17
Gambar 7.5 Penerimaan Payload..... 17
Gambar 7.6 GUI Penerimaan Payload.....17
Gambar 7.7 Pendekripsian Kunci Simetris dan Ciphertext.....18
Gambar 7.8 GUI Pendekripsian Ciphertext..... 18
Gambar 7.9 Verifikasi Keaslian Pesan dan Pengirim..... 19
Gambar 7.10 GUI Verifikasi Keaslian Pesan dan Pengirim.....19

Daftar Tabel

Tabel 6.1 Struktur payload..... 15

1. Pendahuluan

Dalam era pertukaran data digital, keamanan komunikasi antar perangkat dari pengirim hingga tujuan menjadi aspek yang sangat krusial. Pesan yang dikirimkan melalui jaringan yang terbuka sangat rentan terhadap penyadapan/penyerangan oleh pihak-pihak yang tak berwenang. Oleh karena itu, diperlukan sebuah metode keamanan *end-to-end* yang menjamin pesan yang disampaikan hanya dapat dibaca oleh penerima tanpa gangguan pihak eksternal, terjaga keasliannya, dan tidak berubah selama proses transmisi.

Laporan ini bertujuan untuk membahas mengenai implementasi skenario pengiriman pesan aman dari Alice (pengirim) ke Bob (penerima) yang mensimulasikan komunikasi antar alamat IP yang berbeda. Sistem ini dirancang untuk mendemonstrasikan bagaimana berbagai komponen kriptografi bekerja secara harmonis dalam satu alur kerja.

1.1. Enkripsi Simetris

Enkripsi simetris merupakan garda terdepan dalam menjaga kerahasiaan data (confidentiality) karena kemampuannya memproses informasi dalam volume besar dengan efisiensi tinggi. Mekanisme ini bekerja dengan menggunakan satu kunci rahasia yang sama untuk melakukan transformasi data dari plaintext menjadi ciphertext dan sebaliknya. Meskipun sangat cepat, tantangan utamanya terletak pada distribusi kunci; jika kunci tersebut jatuh ke tangan pihak ketiga selama proses transmisi, maka seluruh keamanan pesan akan runtuh. Oleh karena itu, kunci simetris ini tidak boleh dikirimkan dalam bentuk teks biasa di dalam jaringan.

1.2 Enkripsi Asimetris

Untuk mengatasi kelemahan distribusi kunci simetris, digunakanlah enkripsi asimetris yang mengandalkan sepasang kunci matematis yang saling berhubungan, yaitu public key dan private key. Dalam skenario ini, enkripsi asimetris tidak digunakan untuk mengenkripsi isi pesan secara langsung karena keterbatasan kecepatan prosesnya, melainkan untuk mengenkripsi kunci simetris (key wrapping). Dengan menggunakan public key milik Bob, Alice dapat menyegel kunci simetris tersebut sehingga hanya Bob sebagai satu-satunya pemegang private key yang pas yang dapat membukanya kembali. Pola ini memastikan bahwa kunci rahasia dapat berpindah tangan melalui saluran komunikasi yang tidak aman sekalipun tanpa risiko terbaca oleh penyadap.

1.3 Fungsi Hash

Aspek integritas data (integrity) dikelola melalui penggunaan fungsi hash, yang memetakan data masukan menjadi deret karakter unik dengan panjang tetap sebagai sidik jari digital (fingerprint). Sifatnya yang satu arah dan sangat sensitif

terhadap perubahan membuat fungsi ini sangat efektif untuk mendeteksi gangguan. Jika ada modifikasi sekecil apa pun pada pesan selama perjalanan dari Alice ke Bob, nilai hash yang dihitung ulang oleh Bob tidak akan cocok dengan nilai hash yang dikirimkan oleh Alice. Hal ini memberikan jaminan bahwa data yang diterima Bob masih murni dan belum dimanipulasi oleh pihak luar selama proses pengiriman antar IP.

1.4 Digital Signature

Untuk melengkapi arsitektur keamanan, digital signature diterapkan guna menjamin autentikasi pengirim dan non-repudiation. Tanda tangan ini dibuat oleh Alice dengan mengenkripsi nilai hash pesan menggunakan private key miliknya sendiri. Ketika Bob menerima paket tersebut, ia akan memverifikasi tanda tangan itu menggunakan public key milik Alice. Jika verifikasi berhasil, Bob mendapatkan bukti sah bahwa pesan tersebut benar-benar berasal dari Alice (authentication) dan Alice tidak dapat menyangkal pengiriman pesan tersebut karena hanya ia yang memiliki akses ke private key yang bersangkutan.

2. Tujuan Tugas

Tujuan utama dari pengerjaan tugas ini adalah agar mahasiswa dapat memahami dan mengimplementasikan alur pengamanan data secara end-to-end dalam sebuah sistem komunikasi.

2.1. Memahami Alur Keamanan Data End-to-End

Tujuan akhir dari tugas ini adalah agar mahasiswa dapat merancang dan mensimulasikan proses pengamanan data secara menyeluruh mulai dari sisi pengirim (Alice) hingga proses verifikasi di sisi penerima (Bob). Hal ini mencakup pemahaman mendalam tentang bagaimana sebuah pesan yang terkirim tetap terjaga kerahasiaannya, integrasinya hingga keasliannya saat melewati jaringan komunikasi yang tidak aman. Dengan memperhatikan alur ini diharapkan dapat memetakan setiap risiko keamanan yang mungkin terjadi pada setiap tahapan transmisi data.

2.2. Implementasi Kombinasi Komponen Kriptografi

Tugas ini bertujuan untuk memberikan pengalaman teknis dengan mengkombinasikan berbagai algoritma kriptografi yang berbeda untuk mencapai tujuan keamanan yang spesifik. Mahasiswa dituntut untuk mampu mengintegrasikan enkripsi simetris (AES) untuk efisiensi pemrosesan data, enkripsi asimetris (RSA) untuk manajemen kunci serta fungsi hash (SHA-256) dan *digital signature* untuk autentikasi. Melalui tugas ini, diharapkan dapat mengerti bahwa aplikasi *end-to-end* bergantung pada rangkaian serial kriptografi agar aman dari serangan pihak eksternal.

2.3. Simulasi Komunikasi Antar Jaringan (IP)

Mahasiswa mampu mengimplementasikan skenario komunikasi data yang nyata dengan melakukan pengiriman paket dari satu alamat IP sumber ke alamat IP penerima. Hal ini mencangkup kemampuan dalam struktur menyusun *payload* keamanan dalam format JSON, agar dapat ditransmisikan secara konsisten melalui protokol jaringan. Simulasi ini sangat penting untuk memberikan gambaran konkret mengenai tantangan teknis dalam menjaga sinkronisasi data dan format pesan saat berkomunikasi antar perangkat atau logical entity yang berbeda.

2.4. Verifikasi dan Validasi Sisi Penerima

Tugas ini bertujuan untuk melakukan proses dekripsi, kalkulasi ulang *hash* dan verifikasi tanda tangan digital dari sisi penerima secara akurat. Bob harus mampu menyimpulkan secara valid apakah pesan yang diterima telah dimodifikasi, apakah kunci simetris yang dikirimkan autentik, dan apakah identitas pengirim benar-benar terverifikasi. Dengan demikian, mahasiswa dapat membuktikan keberhasilan sistem keamanan yang dibangun melalui hasil pengujian yang objektif dan dapat dipertanggungjawabkan.

3. Desain Skenario

3.1. Entitas dan Topologi Jaringan

Skenario ini melibatkan dua entitas logis utama yang berperan sebagai pengirim dan penerima data, yaitu Alice dan Bob. Komunikasi dilakukan secara *point-to-point* melalui jaringan lokal menggunakan protokol yang mendukung pengiriman paket antar alamat IP. Dalam implementasi ini, Alice bertindak sebagai klien yang menginisiasi pengiriman, sementara Bob bertindak sebagai server yang menunggu kedatangan paket data pada *port* tertentu.

3.2. Alamat IP dan Konektivitas

Setiap entitas diberikan identitas berupa alamat IP unik untuk menyimulasikan lingkungan jaringan yang nyata. Alice dikonfigurasi dengan alamat IP **10.129.119.81**, sedangkan Bob menggunakan alamat IP **10.129.119.120** sebagai tujuan akhir pengiriman. Pemisahan alamat IP ini memastikan bahwa seluruh komponen kriptografi yang dikirimkan harus dikemas dalam satu payload yang utuh agar dapat ditransmisikan melewati socket jaringan tanpa kehilangan data.

3.3. Alur Keamanan Data

Proses pengamanan dimulai saat Alice menyiapkan pesan *plaintext* yang kemudian diamankan melalui beberapa lapis perlindungan kriptografi sebelum

dikirim. Pesan tersebut dienkripsi dengan kunci simetris, lalu kunci simetris tersebut dibungkus menggunakan enkripsi asimetris agar hanya Bob yang bisa membukanya. Selain itu, Alice menambahkan hash dan digital signature ke dalam paket untuk menjamin bahwa isi pesan tidak mengalami perubahan dan identitas pengirim benar-benar valid saat diperiksa oleh Bob.

4. Topologi & Arsitektur Sistem

4.1. Arsitektur Jaringan

Sistem ini menggunakan arsitektur jaringan Host-to-Host, di mana dua perangkat komputer berkomunikasi secara langsung dalam satu segmen jaringan yang sama (Local Area Network). Setiap host diidentifikasi melalui alamat IPv4 unik (dalam pengujian ini menggunakan subnet 10.10.10.0/24), di mana konfigurasi *subnet mask* dan *gateway* dipastikan identik untuk menjamin bahwa paket data yang dikirim Alice dapat mencapai antarmuka jaringan Bob tanpa hambatan *routing* yang kompleks.

4.2. Model Komunikasi *Client-Server*

Interaksi yang terjadi antara Alice dan Bob mengadopsi paradigma komunikasi antara *client* dan *server* untuk melakukan pertukaran data secara sinkron melalui mekanisme *socket programming*. Dalam arsitektur ini, Bob berperan sebagai *Server (Passive Listener)* yang bertugas mengalokasikan sumber daya pada port tertentu (pada simulasi ini digunakan port 5000). Bob melakukan *binding* pada alamat IP 0.0.0.0, yang memungkinkan terjadinya penerimaan data dari antarmuka manapun. Di sisi lain, Alice bertindak sebagai *Client (Active Initiator)* yang memicu jabat tangan tiga arah (*three-way-handshake)*. Alice harus mengetahui IP Bob secara spesifik untuk membangun jalur komunikasi virtual (*tunnel)* sebelum mengirimkan *payload* kriptografi yang telah disusun.

4.3. Komponen Keamanan *End-to-End*

Arsitektur ini menerapkan perlindungan sepenuhnya di tingkat aplikasi sebelum data ditransmisikan ke lapisan transport. Komponen perlindungan utama mencakup manajemen kunci terpisah, di mana setiap pihak memiliki penyimpanan kunci (*key store*) lokal yang berisi pasangan kunci RSA (publik dan privat). Alur data kemudian diamankan menggunakan *Hybrid Cryptosystem Engine* yang menggabungkan efisiensi algoritma simetris dengan kekuatan distribusi kunci asimetris. Selain itu, terdapat komponen *Secure Payload Wrapper* yang berfungsi sebagai pengemas data (Menyatukan ciphertext, kunci simetris terenkripsi, IV, dan tanda tangan digital ke dalam satu struktur objek JSON). Komponen ini menjamin bahwa seluruh parameter keamanan yang dibutuhkan

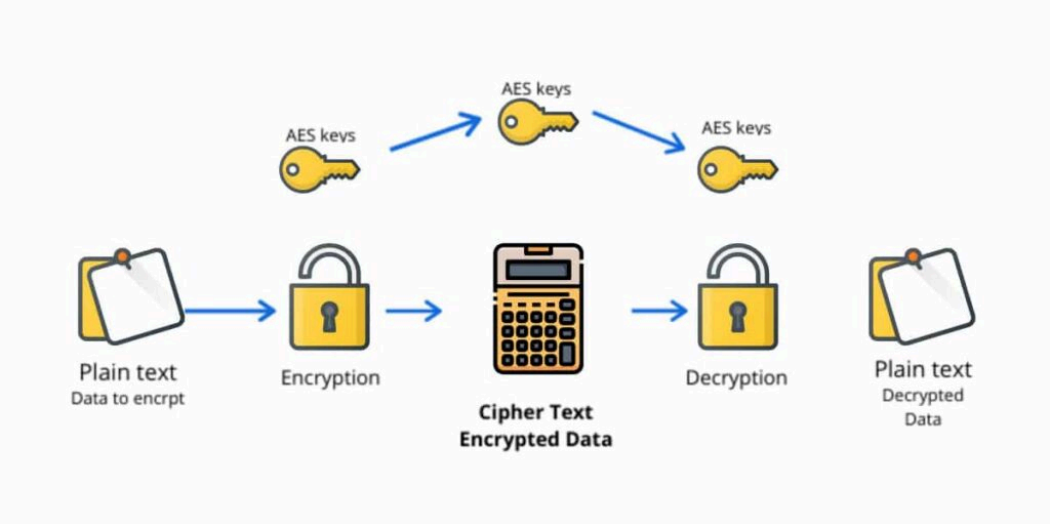
untuk proses dekripsi dan verifikasi di sisi penerima terkirim secara utuh dan terorganisir dalam satu unit transmisi data.

5. Algoritma yang Diterapkan

Sistem keamanan yang dibangun dalam tugas ini menerapkan konsep kriptografi hybrid, yaitu sebuah arsitektur yang menggabungkan efisiensi algoritma simetris dengan keunggulan manajemen kunci pada algoritma asimetris. Pemilihan algoritma ini dilakukan berdasarkan standar industri modern untuk menjamin tingkat keamanan yang optimal dalam pertukaran data antar *host*.

5.1 Enkripsi Simetris: AES-256 (Advanced Encryption Standard)

Untuk mengamankan isi pesan (*plaintext)*, digunakan algoritma AES-256 dengan mode operasi CBC (*Cipher Block Chaining*). Algoritma AES-256 dipilih karena efisiensinya dalam memproses data berukuran besar dengan penggunaan sumber daya komputasi yang relatif rendah. Panjang kunci sebesar 256-bit memberikan perlindungan yang sangat kuat terhadap serangan bertipe *brute force*. Dalam setiap sesi, digunakan *Initialization Vector* (IV) sebesar 16-*byte* yang dihasilkan secara acak untuk memastikan bahwa pesan yang sama tidak akan menghasilkan *ciphertext* yang identik, sehingga meningkatkan ketahanan terhadap serangan pihak ketiga.

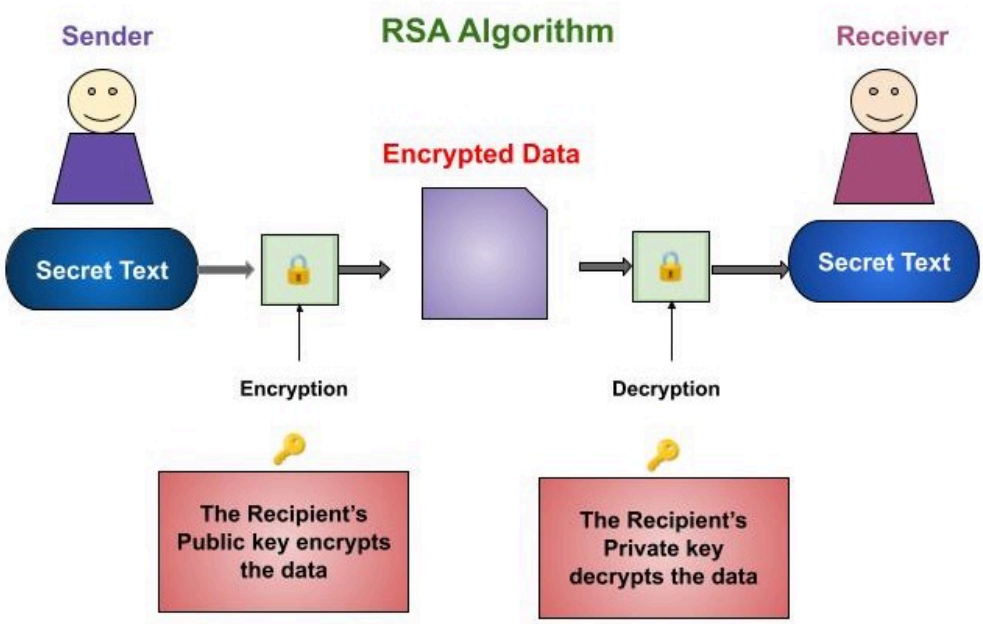


Gambar 5.1 Enkripsi simetris menggunakan AES

5.2 Enkripsi Asimetris: RSA-2048 (Rivest-Shamir-Adleman)

Karena distribusi kunci simetris secara langsung sangat berisiko, digunakan algoritma RSA dengan panjang kunci 2048-bit sebagai mekanisme pertukaran kunci (*key exchange*). Algoritma ini bekerja dengan skema pasangan kunci, yaitu kunci publik dan untuk enkripsi dan kunci privat untuk dekripsi. Dalam skenario ini, Alice menggunakan kunci publik milik Bob untuk

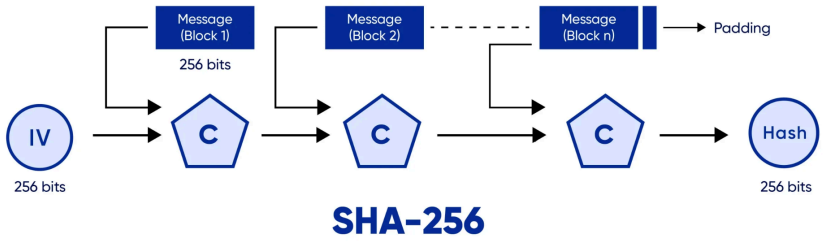
mengenkripsi kunci simetris (AES). Dengan demikian, meskipun paket data disadap di tengah jaringan, hanya Bob yang dapat membuka kunci tersebut menggunakan kunci privat miliknya.



Gambar 5.2 Enkripsi simetris menggunakan RSA

5.3 Fungsi Hash: SHA-256 (Secure Hash Algorithm)

Aspek integritas data dijamin dengan penggunaan fungsi hashing SHA-256. Algoritma ini bertugas menghasilkan nilai digital yang unik dari sebuah pesan. Sifat *one-way* dan *collision-resistance* pada SHA-256 memastikan bahwa perubahan sekecil apapun pada isi pesan akan menghasilkan nilai hash yang berbeda. Nilai hash ini yang kemudian menjadi dasar bagi pembentukan tanda tangan digital untuk memvalidasi bahwa data tidak dimodifikasi selama proses pengiriman.



Gambar 5.2 *Hash* menggunakan RSA

5.4 Digital Signature: RSA-PSS (Probabilistic Signature Scheme)

Untuk menjamin aspek autentikasi dan *non-repudiation*, diterapkan mekanisme *Digital Signature* menggunakan standar RSA-PSS. Alice menandatangani pesan menggunakan kunci privat miliknya, dan Bob memverifikasi menggunakan kunci publik Alice. Penggunaan skema PSS dipilih karena sifatnya yang probabilistik dengan penambahan *salt* acak, yang memberikan tingkat keamanan lebih tinggi terhadap pemalsuan tanda tangan. Mekanisme ini membuktikan secara sah bahwa pesan tersebut benar-benar dikirim oleh Alice.

6. Langkah Kerja & Implementasi

6.1. Sisi Alice (Pengirim)

Pada sisi pengirim, Alice bertanggung jawab untuk menginisiasi seluruh proses enkripsi dan tanda tangan digital. Langkah-langkah yang dilakukan meliputi:

1. **Pemuatan Kunci**

   Alice memuat kunci privat miliknya sendiri untuk melakukan penandatanganan digital dan kunci publik milik Bob melalui jalur komunikasi yang aman (dalam simulasi dilakukan pertukaran manual).

```
ROOT_DIR = Path(__file__).resolve().parents[2]
KEYS_DIR = ROOT_DIR / "keys"

with open(KEYS_DIR / "alice_private.pem", "rb") as f:
    private_key_alice =
serialization.load_pem_private_key(f.read(),
password=None)
with open(KEYS_DIR / "bob_public.pem", "rb") as f:
    public_key_bob =
serialization.load_pem_public_key(f.read())
```

2. **Pembuatan Kunci Sesi**

   Sistem menghasilkan kunci asimetris AES 256-bit dan *Initialization Vector* (IV) secara acak. Kunci ini bersifat *ephemeral* (hanya untuk satu kali sesi).

3. **Enkripsi Data**

   Pesan asli (*plaintext*) diberikan padding agar sesuai dengan ukuran blok AES, kemudian dienkripsi menggunakan mode CBC (*Cipher Block Chaining*).

```
sym_key = os.urandom(32)   # 32 byte → 256-bit AES session
```

```
key (ephemeral)
iv = os.urandom(16)        # 16 byte IV untuk AES-CBC
padder = sym_padding.PKCS7(128).padder()
padded_data = padder.update(plaintext) + padder.finalize()
cipher = Cipher(algorithms.AES(sym_key), modes.CBC(iv))
encryptor = cipher.encryptor()
ciphertext = encryptor.update(padded_data) +
encryptor.finalize()
```

4. **Enkripsi Kunci**

   Kunci AES yang digunakan untuk mengenkripsi pesan kemudian dikunci kembali menggunakan kunci publik milik Bob. Hal ini memastikan hanya Bob yang bisa mengakses kunci tersebut.

```
encrypted_sym_key = public_key_bob.encrypt(
    sym_key,

asym_padding.OAEP(mgf=asym_padding.MGF1(hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)
)
```

5. **Penandatanganan Digital**

   Alice membuat *hash* dari *plaintext* menggunakan SHA-256 dan menandatanganinya dengan *Private Key* miliknya menggunakan skema RSA-PSS.

```
signature = private_key_alice.sign(
    plaintext,

asym_padding.PSS(mgf=asym_padding.MGF1(hashes.SHA256()),
salt_length=asym_padding.PSS.MAX_LENGTH),
    hashes.SHA256()
)
```

6. **Transmisi**

   Seluruh komponen dikonversi ke format Base64, disusun ke dalam objek JSON, dan dikirimkan melalui *socket* TCP ke alamat IP Bob pada port 5000.

```
payload = {
    'ciphertext': base64.b64encode(ciphertext).decode(),
    'encrypted_key':
base64.b64encode(encrypted_sym_key).decode(),
    'iv': base64.b64encode(iv).decode(),
```

```
    'signature': base64.b64encode(signature).decode()
}

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
s:
    s.connect((BOB_IP, PORT))
    s.sendall(json.dumps(payload).encode())
    print("Pesan telah dikirim ke Bob.", flush=True)
```

6.2. Sisi Bob (Penerima)

Sisi penerima dirancang untuk bekerja secara pasif dan melakukan validasi menyeluruh terhadap paket yang diterima. Langkah-langkah yang dilakukan meliputi:

1. **Socket Listening**

   Bob menjalankan layanan pada port 5000 dan menunggu koneksi dari Alice. Setelah koneksi terbentuk, Bob menerima paket JSON dan melakukan dekode dari format base64 kembali ke bentuk *byte*.

```
def start_bob():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM)
as s:
        s.bind(('0.0.0.0', PORT))       # listen di port
5000
        s.listen()
        print(f"Bob menunggu pesan di port {PORT}...",
flush=True)
        conn, addr = s.accept()         # tunggu koneksi
dari Alice
        with conn:
            print(f"Terhubung dengan IP Alice: {addr}",
flush=True)
            data = conn.recv(8192)      # terima payload
JSON mentah
            payload = json.loads(data.decode())
            pretty_payload = json.dumps(payload, indent=2)
            print(f"Payload JSON:\n{pretty_payload} \n\n",
flush=True)

            # Dekode Base64 ke bytes
            enc_sym_key =
base64.b64decode(payload['encrypted_key'])
            ciphertext =
base64.b64decode(payload['ciphertext'])
            iv = base64.b64decode(payload['iv'])
            signature =
```

```
base64.b64decode(payload['signature'])
```

2. **Dekripsi Kunci Simetris**

   Bob menggunakan kunci privat miliknya untuk membuka kunci AES. Jika kunci ini gagal dibuka, maka proses dihentikan dengan asumsi kunci salah atau data korup.

```
sym_key = private_key_bob.decrypt(
    enc_sym_key,
    asym_padding.OAEP(
        mgf=asym_padding.MGF1(hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
```

3. **Dekripsi Pesan**

   Dengan kunci AES yang telah dibuka, Bob melakukan dekripsi terhadap *ciphertext*. Setelah proses dekripsi selesai, dilakukan *unpadding* untuk mengembalikan pesan menjadi *plaintext*.

```
cipher = Cipher(algorithms.AES(sym_key), modes.CBC(iv))
decryptor = cipher.decryptor()
padded_plaintext = decryptor.update(ciphertext) +
decryptor.finalize()
unpadder = sym_padding.PKCS7(128).unpadder()
plaintext = unpadder.update(padded_plaintext) +
unpadder.finalize()

print(f"\nPesan Diterima: {plaintext.decode()}",
flush=True)
```

4. **Verifikasi Autentikasi**

   Bob menggunakan kunci publik milik Alice untuk memverifikasi tanda tangan digital yang disertakan dalam *payload*. Proses ini membandingkan *hash* pesan yang diterima dengan *hash* yang ada dalam tanda tangan.

```
public_key_alice.verify(
    signature, plaintext,
    asym_padding.PSS(
        mgf=asym_padding.MGF1(hashes.SHA256()),
        salt_length=asym_padding.PSS.MAX_LENGTH
    ),
```

```
    hashes.SHA256()
)
print("Hasil: [VERIFIED] Pesan asli dari Alice dan belum
diubah.", flush=True)
```

6.3. Format Payload

Penggunaan JSON memungkinkan data yang kompleks (seperti kunci dan tanda tangan) dikirimkan secara terorganisir dalam satu aliran data tunggal. Berikut adalah struktur *payload* yang digunakan:

Tabel 6.1 Struktur payload

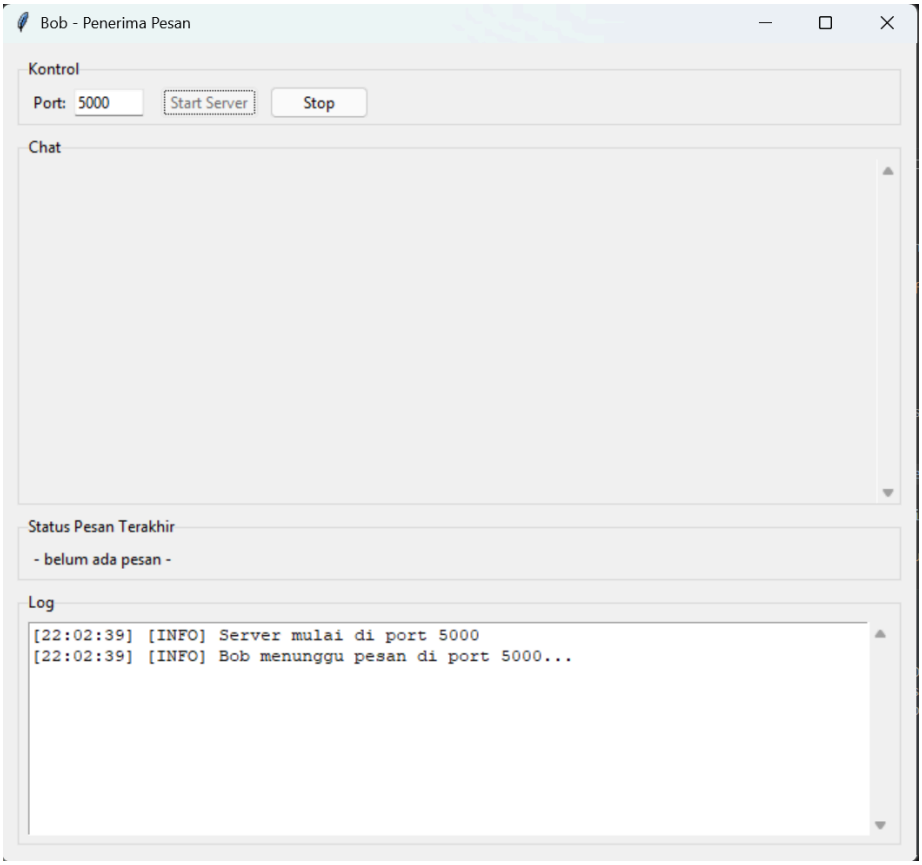
Key	Deskripsi	Fungsi Keamanan
ciphertext	Pesan asli yang telah dienkripsi AES-256	Confidentiality (Kerahasiaan)
encrypted_key	Kunci AES yang dibungkus RSA Public Key Bob	Secure Key Exchange
IV	Nilai acak untuk inisialisasi blok pertama AES	Non-deterministic Encryption
digital_signature	Hasil signing pesan oleh Private Key Alice	Authentication & Integrity

7. Hasil Pengujian & Analisis

Pengujian dimulai dengan mengaktifkan Bob sebagai Server yang dikonfigurasi untuk memantau permintaan masuk melalui port 5000. Dengan melakukan *binding* pada alamat IP 0.0.0.0, Bob mampu menerima data dari antarmuka jaringan manapun yang tersedia secara *real-time*. Pengaturan ini memastikan bahwa sistem siap menerima dan memproses *payload* kriptografi dari Alice segera setelah koneksi TCP terbentuk.

```
PS C:\Users\w2309\end-to-end> python src/Bob/bob.py
Bob menunggu pesan di port 5000...
```

Gambar 7.1 Pemantauan Permintaan Masuk

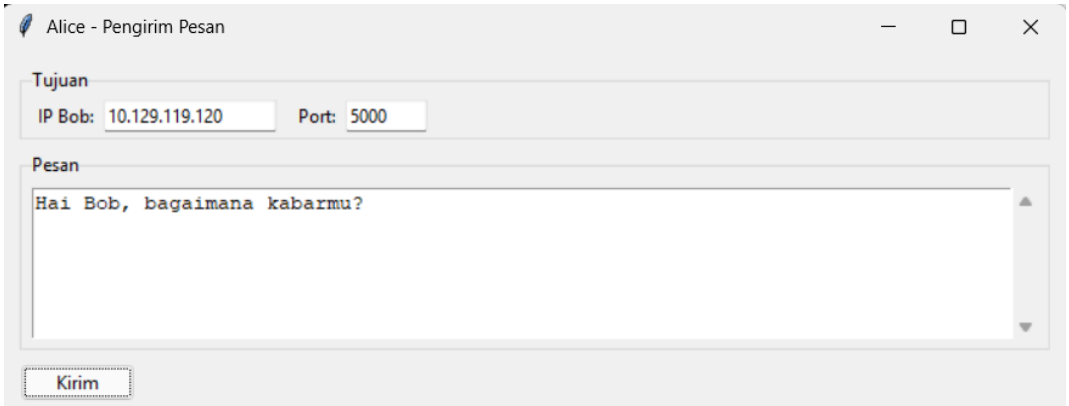


Gambar 7.2 GUI Pemantauan Permintaan Masuk

Langkah selanjutnya dilakukan oleh Alice sebagai Client yang bertindak sebagai inisiator pengiriman data melalui jaringan. Sebelum transmisi dilakukan, pesan (*plaintext*) melewati serangkaian proses kriptografi berlapis, mulai dari enkripsi simetris AES untuk isi pesan hingga enkripsi asimetris RSA untuk mengamankan kunci sesi. Setelah seluruh komponen keamanan dibungkus dalam format *payload* JSON, Alice mengirimkan data tersebut langsung ke alamat IP dan port yang telah dibuka oleh Bob.

```
PS C:\Users\Lenovo\end-to-end> python src/Alice/alice.py
Masukkan pesan untuk Bob: Hai Bob, bagaimana kabarmu?
Pesan telah dikirim ke Bob.
```

Gambar 7.3 Pengiriman Pesan oleh Alice

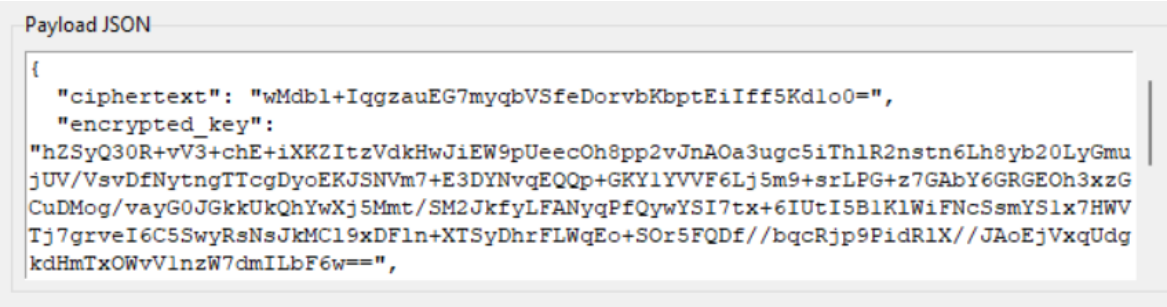


Gambar 7.4 GUI Pengiriman Pesan oleh Alice

Proses penerimaan di sisi Bob menandai dimulainya fase dekapsulasi data, di mana koneksi TCP yang stabil memungkinkan transfer seluruh komponen keamanan secara utuh. Setelah jabat tangan (*three-way-handshake*) jaringan berhasil, Bob mengekstrak struktur JSON yang dikirimkan oleh Alice untuk memisahkan setiap elemen kriptografi ke dalam memori lokal.


```
Bob menunggu pesan di port 5000...
Terhubung dengan IP Alice: ('10.129.119.81', 53274)
Payload JSON:
{
  "ciphertext": "tRNAMeTTt37yhFu0SorekLcbD2ir+aE+XBjGFYLgybQ=",
  "encrypted_key": "fdLfaapaEgRSjGK4ueDyDWFCVw++gzC0SpDqSdekzfuKt5Kl5sTXVwIIfptBuMXhzKQj16/KmuUOMEEfOpHYzjJiActtD01ZcI
xhjVzSBt7MsKkjKnBRpt8+XhQqMK+BHcxSc5wQcy2OLsPsNsTc19yRMjVn4oQjpr9cFrEm/cRBIlAu3eOIuCJqMTWJ5OUiZ8q54XoHn63crkFFP4sc/AzX
vp2cW3j76Sxnyef2r9TAgfJrNO49LnWSKauWJiDG2f9dhqGj96ULXenekM2R3uzRsx1k+vyq8QZnnch/P7eR7g1Ir3uyNSG4w9FQ1581kPGhR0FGv/sn8R
TyqPH8VQ==",
  "iv": "6EbqvULsX3v+Ob00Fs981g==",
  "signature": "IvXPTMQp55F8x6n0JMlEgQIKYpHnaQNnsJxMgE8F48VEX+4rL6KgwN2K8oqsroQrgzD0E2E9O65Iy4N4Ei5FuM+1e/1azdBxBMrB5i
IFcUX9/FpXPDtN2mVXFWBUDmya104yGOyTPjyfatszCnAXAj+ZNjd68qCsjqX4j5Rv0lBeSYveWNFAT1aq66cSFbPLAY1/PKjd783yjsM2VqAuDauNb3jd
hn+2qxyW04fV8Jj6wuqHeM5OfwdgRxClEzrMwIFdGJdzYDzBHKAprgRmt0ZWC/TxNwEWzPjeHX50iMOngT38a6FIwKKRLWV0QAroaD9qWbmfPpIRRAKBBK
bdyw=="
}
```


Gambar 7.5 Penerimaan *Payload*



Gambar 7.6 GUI Penerimaan *Payload*

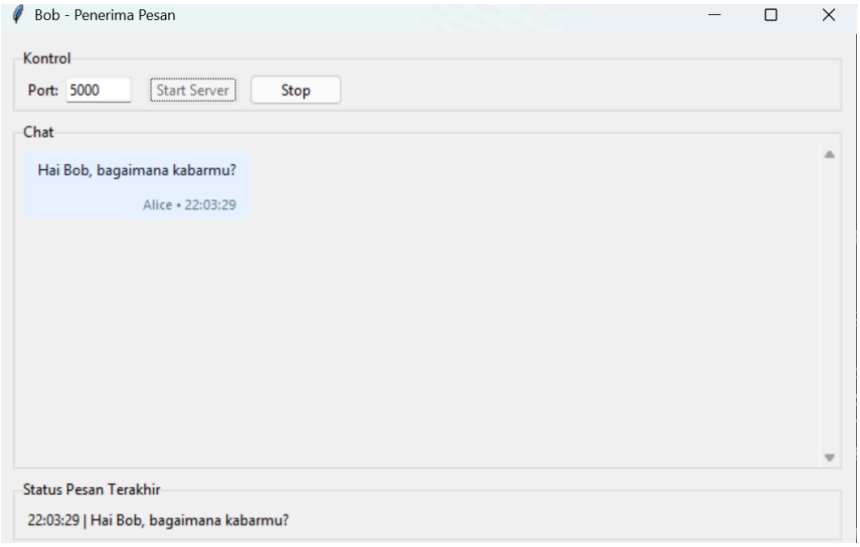
Tahap berikutnya adalah proses dekripsi kunci simetris (*encrypted_key*) menggunakan *Private Key* milik Bob, yang merupakan satu-satunya kunci yang mampu membuka bungkusan asimetris tersebut. Keberhasilan tahap ini sangat krusial, karena tanpa kunci simetris yang valid, seluruh rangkaian data lainnya tidak akan dapat diproses.

Setelah kunci simetris (AES *key*) berhasil dipulihkan, kunci tersebut langsung digunakan oleh mesin dekripsi AES untuk memproses *ciphertext*. Dengan bantuan *Initialization Vector* (IV) yang disertakan dalam *payload*, algoritma akan melakukan transformasi data hingga kembali menjadi pesan asli (*plaintext*) yang dapat dibaca. Proses ini memastikan bahwa meskipun data sempat berada dalam bentuk acak, penerima yang sah dapat mengembalikannya secara sempurna tanpa kehilangan informasi sedikit pun.

```
[DEBUG] Kunci simetris didekripsi

Pesan Diterima: Hai Bob, bagaimana kabarmu?
```

Gambar 7.7 Pendekripsian Kunci Simetris dan *Ciphertext*



Gambar 7.8 GUI Pendekripsian *Ciphertext*

Tahap terakhir adalah memastikan bahwa pesan tidak diubah selama proses transmisi menggunakan digital signature. Proses ini meliputi keaslian pesan dan keaslian pengirim. Keaslian pengirim diperiksa menggunakan publik key Alice yang dipegang oleh Bob, dan apabila pesan dimodifikasi pihak ketiga (*Man-in-the-middle*), maka sistem otomatis akan menghasilkan status gagal.

```
Pesan Diterima: Hai Bob, bagaimana kabarmu?
Hasil: [VERIFIED] Pesan asli dari Alice dan belum diubah.
```

Gambar 7.9 Verifikasi Keaslian Pesan dan Pengirim

```
Log
[22:02:39] [INFO] Server mulai di port 5000
[22:02:39] [INFO] Bob menunggu pesan di port 5000...
[22:03:29] [INFO] Pesan didekripsi: Hai Bob, bagaimana kabarmu?
[22:03:29] [VERIFIED] Tanda tangan valid dari Alice
```

Gambar 7.10 GUI Verifikasi Keaslian Pesan dan Pengirim

8. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem komunikasi yang aman antara Alice dan Bob, dapat ditarik beberapa kesimpulan:

1. Sistem telah mampu memenuhi empat aspek keamanan utama, yaitu *Confidentiality* (pesan tidak dapat dibaca tanpa kunci privat yang sah), *Integrity* (Data yang diterima identik dengan data yang dikirim), *Authentication* (Identitas pengirim tervalidasi melalui tanda tangan digital), dan *Non-repudiation* (Pengirim tidak dapat menyangkal pesan karena penggunaan kunci privat yang unik).
2. Penggabungan algoritma simetris AES-256 dan asimetris RSA-2048 terbukti menjadi solusi yang efisien. AES-256 memberikan kecepatan dalam pemrosesan data pesan, sementara RSA-2048 menjamin keamanan distribusi kunci simetris tanpa risiko penyadapan di jaringan terbuka.
3. Penggunaan RSA-PSS sebagai standar *Digital Signature* memberikan perlindungan tingkat tinggi terhadap manipulasi data. Pengujian menunjukkan bahwa perubahan pada *ciphertext* dapat menyebabkan kegagalan verifikasi, sehingga menjamin keaslian informasi yang diterima.